

EcoGrid : Autonomous Smart Grid Database

CS232-Database Management Systems

Instructor : Ahsan Shah



Name	Registration Number
Hasnat Nawaz	2024223
Syed Rayyan Hassan	2024621
Hamna Qamar	2024198
Syeda Fatima Tu Zahra	2024625

ABSTRACT

EcoGrid is an autonomous Database Management System designed to manage the backend operations of a modern smart grid. Acting as a Meter Data Management System (MDMS) built on PostgreSQL, the system ingests high-frequency telemetry data from digital smart meters.

By utilizing database triggers, stored procedures, and scheduled jobs, EcoGrid automates complex tasks such as time-of-use billing and load monitoring directly within the database layer. This approach provides a highly efficient, software-driven solution that replaces traditional manual electricity reading and billing processes.

The system integrates a real-time dashboard powered by WebSockets, enabling continuous monitoring of electricity usage and system load. Additionally, caching mechanisms using Redis improve performance and responsiveness. The platform is containerized using Docker to ensure scalability, portability, and ease of deployment.

BRIEF INTRODUCTION

The current power distribution network heavily relies on manual meter reading. Personnel must physically visit locations, or consumers must manually upload meter readings to calculate monthly usage. This process is inefficient, prone to human error, and often results in delays in billing and revenue collection.

With the transition toward Advanced Metering Infrastructure (AMI), there is a growing demand for robust backend systems capable of processing massive volumes of real-time data generated by smart meters.

EcoGrid addresses this need by providing a database-centric solution. Instead of relying on external application servers to perform core computations, EcoGrid leverages PostgreSQL's advanced features such as triggers, stored procedures, and scheduling extensions to create an autonomous processing environment.

The system continuously processes incoming data, monitors electricity load, and generates accurate invoices in near real-time. This reduces operational overhead, minimizes errors, and ensures efficient and timely billing for both providers and consumers.

SYSTEM FEATURES

Automated High-Frequency Telemetry Ingestion

The system efficiently handles continuous, high-volume data streams from digital smart meters without degrading database performance.

Native Database Analytics

Time-series aggregation and analytical computations are executed within the database layer, enabling fast insights without impacting transactional operations.

Autonomous Time-of-Use (ToU) Billing

Database triggers and scheduled jobs monitor electricity consumption against tariff rules. Bills are generated automatically without human intervention.

Real-Time Load Dashboard

A React-based dashboard integrated with WebSockets provides live visualization of electricity consumption and system load.

High-Performance Caching Layer

Redis is used to cache frequently accessed data, significantly improving response times for real-time dashboards.

Scalable and Containerized Deployment

The system is containerized using Docker, ensuring portability, scalability, and simplified deployment across environments.

TECH STACK & SELECTION MOTIVATION

The selected technologies are based on their proven effectiveness in handling large-scale data processing and smart grid infrastructures.

Core Database: PostgreSQL

PostgreSQL is a powerful relational database system with strong support for analytical queries, extensibility, and reliability. Research indicates that relational databases enhanced with analytical capabilities are highly effective for smart meter data processing and billing workflows.

Database Extension: TimescaleDB and pg_cron

Time-series data generated by smart meters requires specialized handling. TimescaleDB enables efficient storage and querying through hypertables, significantly improving performance for time-series workloads. pg_cron allows automated scheduling of database tasks such as billing cycles.

Backend & Data Processing: Node.js

Node.js is used to handle asynchronous operations, generate continuous smart meter data streams, and provide an API layer for communication between the database and frontend systems.

Frontend: React.js with WebSockets

React.js enables the development of dynamic and responsive user interfaces. WebSockets facilitate real-time communication for live data visualization.

Caching Layer: Redis

Redis is used to cache frequently accessed data, reducing database load and improving performance for real-time applications.

Containerization: Docker

Docker ensures consistent environments for development and deployment, improving scalability and simplifying system management.

PROJECT DEVELOPMENT PLAN (SDLC)

The project will follow a structured Software Development Life Cycle (SDLC) consisting of the following phases:

Requirement Analysis

Identify system requirements, define project scope, and understand functional and non-functional needs such as performance, scalability, and real-time processing.

System Design

Design the overall system architecture, including database schema, data flow, and component interactions. Define ER models and select appropriate technologies.

Development

Develop the database structure, backend logic, and frontend interface. Implement features such as data ingestion, billing automation, and dashboards.

Integration

Integrate all components including database, backend services, caching layer, and frontend interface to ensure seamless communication between modules.

Testing

Perform unit testing, integration testing, and system testing to ensure correctness, performance, and reliability of the system.

Deployment

Deploy the system using Docker containers in a controlled environment, ensuring proper configuration and scalability.

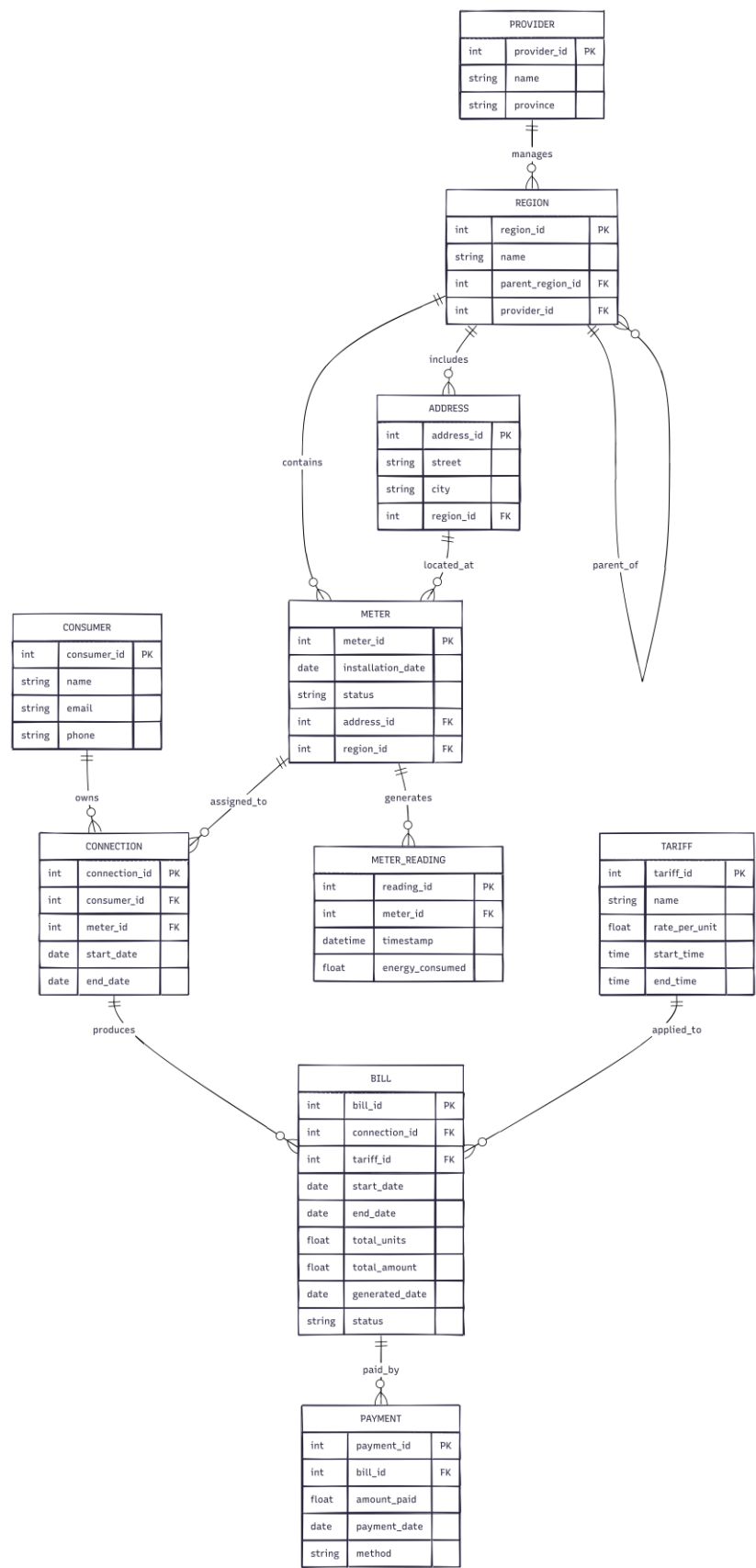
Maintenance & Optimization

Monitor system performance, fix bugs, and optimize queries and processes for improved efficiency over time.

Phase	Week					
	April				May	
	1	2	3	4	1	2
Requirement Analysis						
System Design						
Development						
Integration						
Testing						
Deployment						
Maintenance & Optimization						

Table 5.1 Project Development Plan

ER MODEL



VISION & PROJECT JUSTIFICATION

1. What is your product?

EcoGrid is an autonomous smart grid database system that manages real-time electricity usage data, performs automated billing, and provides analytics for energy distribution.

2. Whom is it for?

The system is designed for electricity distribution companies, utility providers, and energy management organizations that require efficient and automated data processing systems.

3. What problem does it solve?

It eliminates the inefficiencies of manual meter reading and delayed billing by providing automated, accurate, and real-time data processing and invoice generation.

4. What alternatives are available?

Traditional systems rely on manual readings or external application servers for processing. Some modern solutions use cloud-based analytics platforms, but they often depend heavily on external processing layers.

5. Why is this project compelling and worth developing?

The increasing adoption of smart meters requires efficient backend systems. EcoGrid provides a scalable and automated solution that reduces operational costs, improves accuracy, and enhances real-time decision-making.

6. Objectives, Differentiators, Target Customers, and Scope

- **Objectives:** Automate billing, process real-time data, and provide analytical insights.
- **Differentiators:** Database-centric processing using triggers and scheduling, reducing reliance on external systems.
- **Target Customers:** Utility companies and smart grid operators.
- **Scope:** Backend data management, billing automation, and real-time monitoring dashboards.

7. Competitors and Novelty

Existing solutions include enterprise smart grid management systems that rely heavily on distributed application layers. EcoGrid's novelty lies in shifting core processing directly into the database layer, improving efficiency, reducing system complexity, and enabling faster data processing.

